



# Introducción al SoC Zynq-7000 (Zybo Z7-10)

Lab. Electrónica Digital II

David Orlando Briceño González

[dbriceno@unal.edu.co](mailto:dbriceno@unal.edu.co)

# Contenido

## Lab 04 – Procesador Block Design



- 1) ¿Qué es un SoC?
- 2) FPGA tradicional vs SoC
- 3) ¿Que tiene la Zybo Z7 – 10?
- 4) Arquitectura del Zynq – 7000
- 5) ¿Qué significa cada bloque del diagrama?
- 6) ¿Cómo lo usaremos en el laboratorio?
- 7) Preguntas finales

# ¿Qué es un SoC?

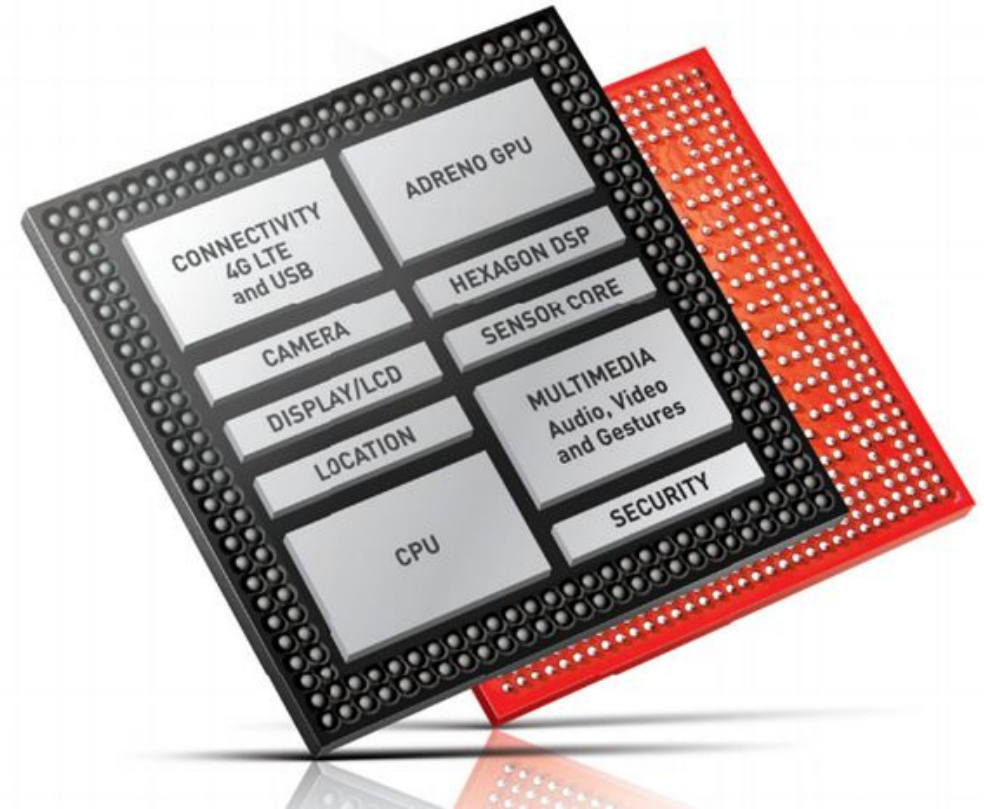
## System on Chip



Es un sistema computacional completo dentro de un solo chip.

Integra:

- Procesador (CPU)
- Memoria / controladores de memoria
- Interfaces de comunicación
- Temporizadores
- GPIO
- Buses internos
- A veces aceleradores de software



# ¿Por qué importa un SoC?

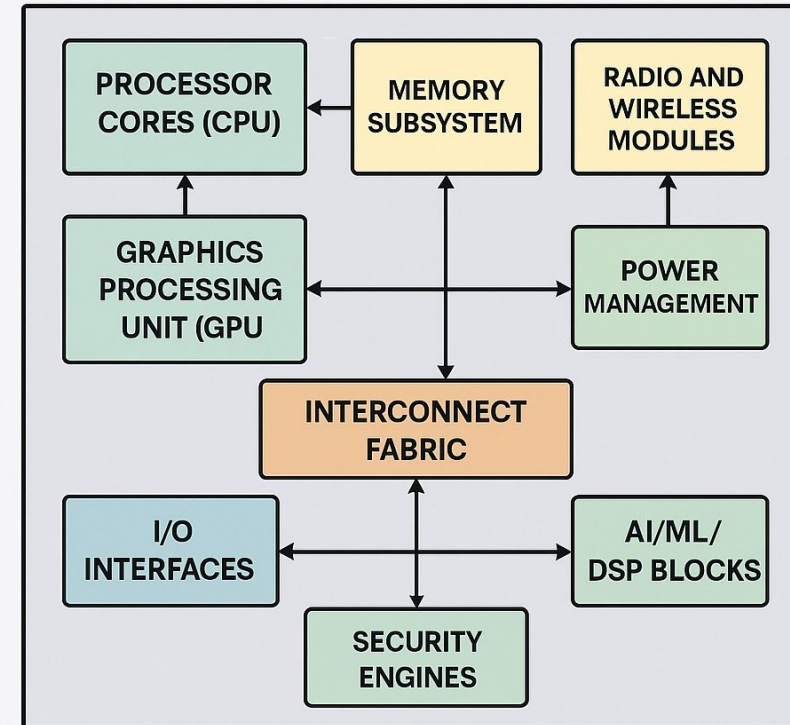
## System on Chip

### Ventajas:

- Menor tamaño
- Menor consumo
- Mayor velocidad interna
- Menor costo
- Mayor integración
- Más confiabilidad

Permite construir productos reales.

## SYSTEM ON CHIP (SoC)



# FPGA tradicional vs SoC

## System on Chip



### FPGA tradicional – solo lógica programable

Usted diseña:

- FSM
- ALU
- Controladores
- UART
- Procesadores softcore si quiere CPU

### SoC FPGA

Tiene:

- CPU real integrada
- FPGA integrada
- Comunicación integrada

**La Zybo es un computador + una FPGA juntos**

# ¿Qué tiene la Zybo Z7-10?

AMD/Xilinx Zynq-7000 XC7Z010



## Esto significa:

Usted diseña:

- Doble núcleo ARM Cortex A9
- FPGA Serie 7
- DDR3 externa en tarjeta
- USB
- Ethernet
- VGA/HDMI
- Switches, LEDs, botones
- PMODs

## ¿Qué implica?

### Podemos programar:

En hardware. Con Verilog/VHDL en Vivado.

### En software

Con C/C++ en Vitis.

**La Zybo es un computador + una FPGA juntos**

# Dos mundos dentro de un chip

AMD/Xilinx Zynq-7000 XC7Z010



## Processing System (PS)

Es el procesador ARM.

Corre:

- C
- Bare metal
- RTOS
- Linux

## Programmable Logic (PL)

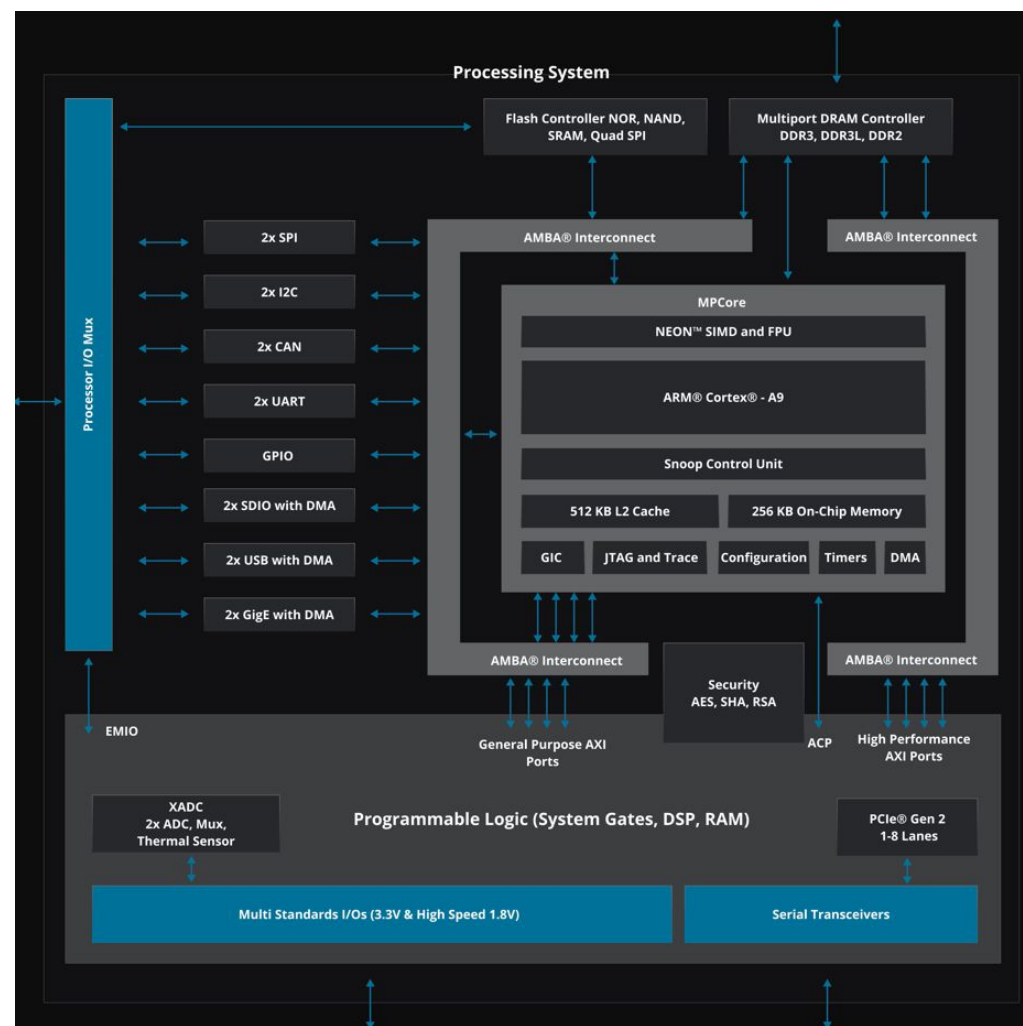
Es la FPGA.

Implementa:

- ALU
- UART custom
- PWM
- Filtros digitales
- Procesadores propios
- Interfaces especiales

**PS piensa.  
PL acelera.**

# Arquitectura general ZYNQ-7000





# Processing System

## ARM Cortex-A9 – CPU principal

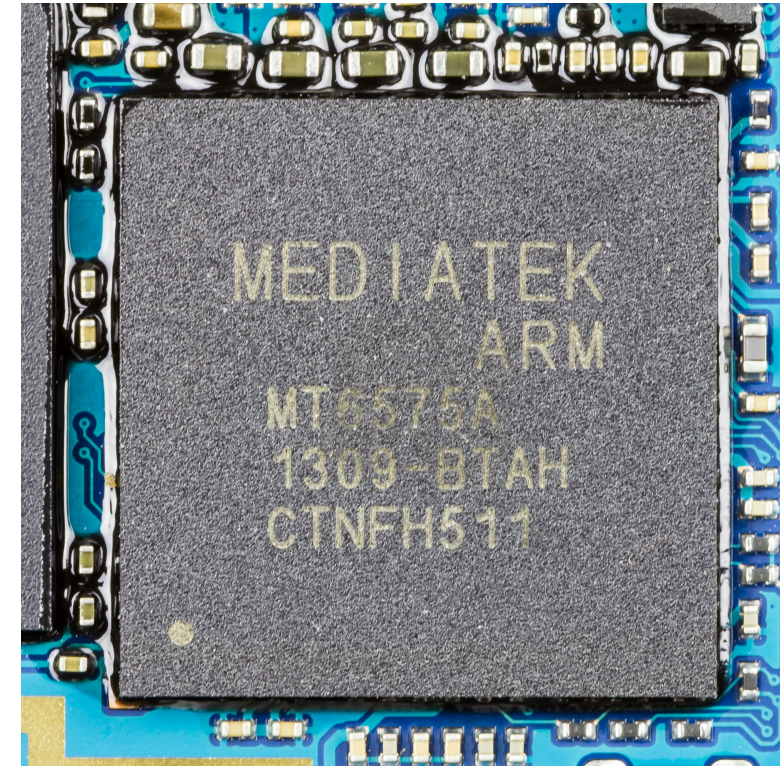
### Ejecuta instrucciones:

C

- If()
- While()
- Printf()
- Variables

### Dual Core

- Corre programas
- Toma decisiones
- Controla periféricos



| Developer | Name      | Cores | Process | NEON SIMD | Vector floating point unit |
|-----------|-----------|-------|---------|-----------|----------------------------|
| Xilinx    | Zynq-7000 | 2     | 28 nm   | Yes       | VFPv3                      |

# Caché y memorias

## ARM Cortex-A9 – CPU principal

### L1 Cache

Memoria ultra rápida dentro del CPU

### L2 Cache (512 KB)

Reduce accesos lentos a RAM

On-chip memory (256 KB)

RAM interna rápida

CPU necesita datos rápido

La caché evita esperar memoria externa



# DDR Controller

ARM Cortex-A9 – CPU principal

**Multiport DRAM Controller**  
Controla memoria DDR externa.

En la tarjeta Zybo hay RAM física externa.

El procesador Guarda:

- Variables grandes
- Programas
- Buffers
- Sistema operativo

**DDR**



**DDR2**



**DDR3**



**DDR4**



**DDR5**



# Periféricos del PS

## ARM Cortex-A9 – CPU principal

**SPI** – Comunicación serial rápida con memorias o sensores

**I2C** – Sensores, RCT, ADC externos.

**UART** – Puerto serial consola.

**CAN** – Automotriz/industrial

**GPIO** – Pines digitales

**USB** – Mouse, teclado, memoria.

**GigE** – Ethernet Gigabit





# DMA – Direct Memory Access

ARM Cortex-A9 – CPU principal



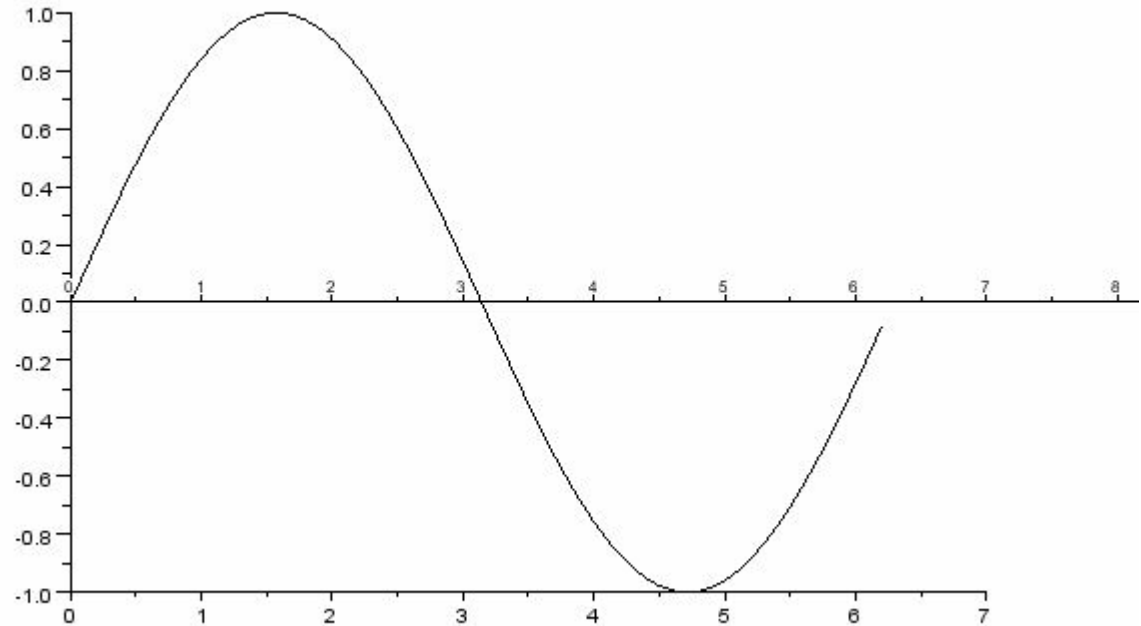
## Direct Memory Access

Mueve datos sin cargar al CPU.

Ejemplo:

- ADC – memoria directa.

Muy útil en alto desempeño



# Timers

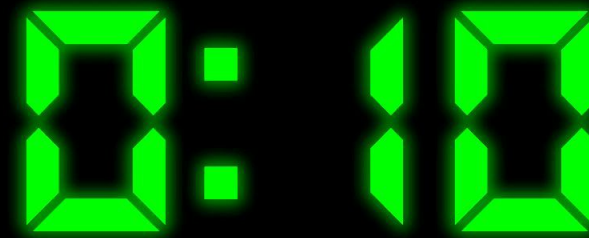
ARM Cortex-A9 – CPU principal



## Temporizadores internos.

Sirven para:

- Delays precisos
- Interrupciones periódicas
- PWM base software
- Medición de tiempo



# JTAG / Trace

ARM Cortex-A9 – CPU principal

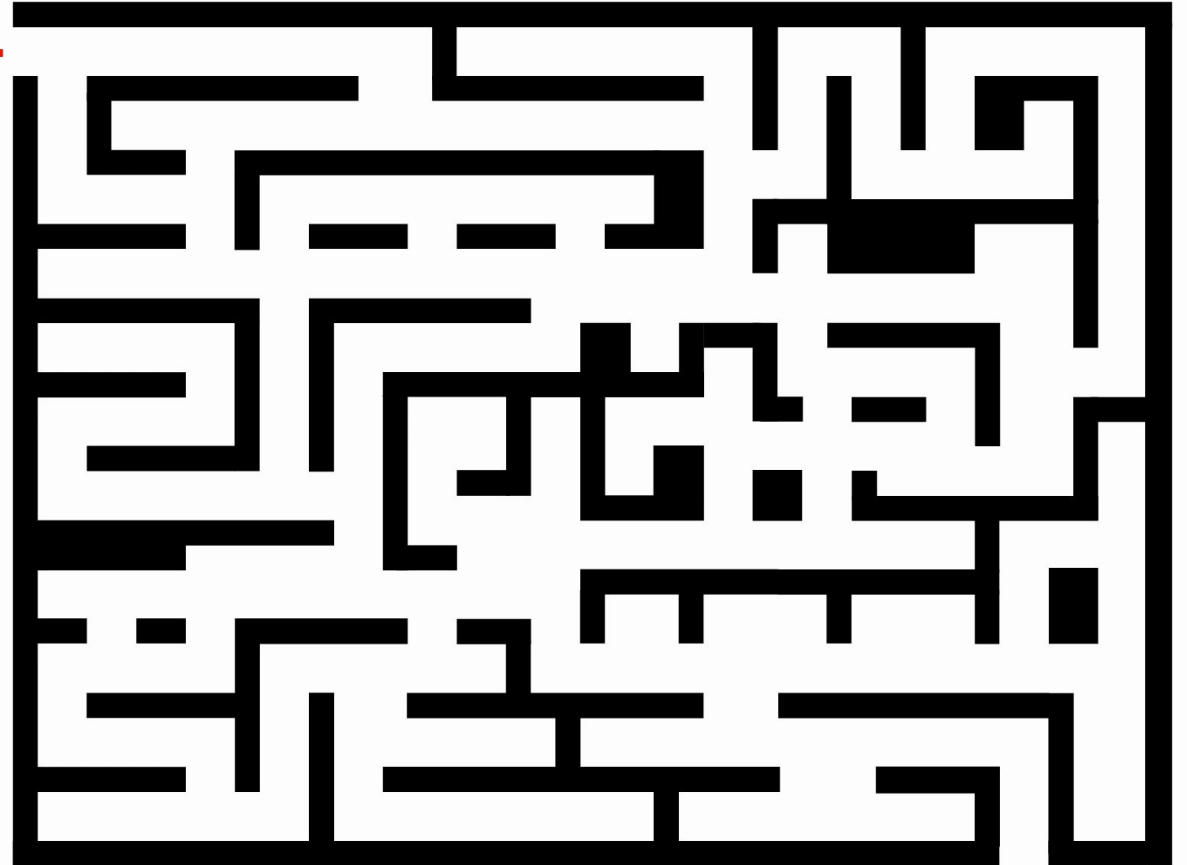


## JTAG

Programación y depuración

## Trace

Ver qué ejecuta CPU internamente



# Programmable Logic (PL)

*ARM Cortex-A9 – aquí vive la FPGA*



Recursos:

- LUTs
- Flip-flops
- DSP Slices
- BRAM
- Clock Managers
- Routing



# XADC – AXI Ports



*ADC interno del chip – bus de comunicación.*

Convierte señales analógicas a digital.

Puede leer:

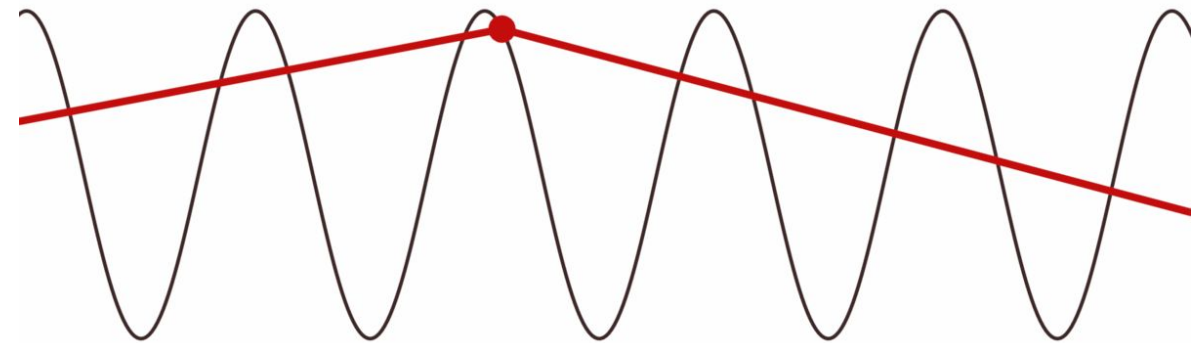
- Voltajes externos
- Temperatura interna

AXI = bus de comunicación

Conecta PS con PL.

Tipos:

- GP – General purpose: CPU controla periféricos FPGA
- HP – High Performance: transferencia rápida de datos
- ACP: acceso coherente con caché.



— actual input  
— captured waveform

En el laboratorio: usaremos GP para GPIO y  
periféricos simples



# Del Hardware Programable al Sistema Computacional Completo

Laboratorio de Electrónica Digital II

David Orlando Briceño González

[dbriceno@unal.edu.co](mailto:dbriceno@unal.edu.co)

# ¿Qué hemos construido hasta ahora?



Compuertas lógicas



Máquinas de estados (FSM)



ALU



Banco de registros



Sistemas digitales modulares

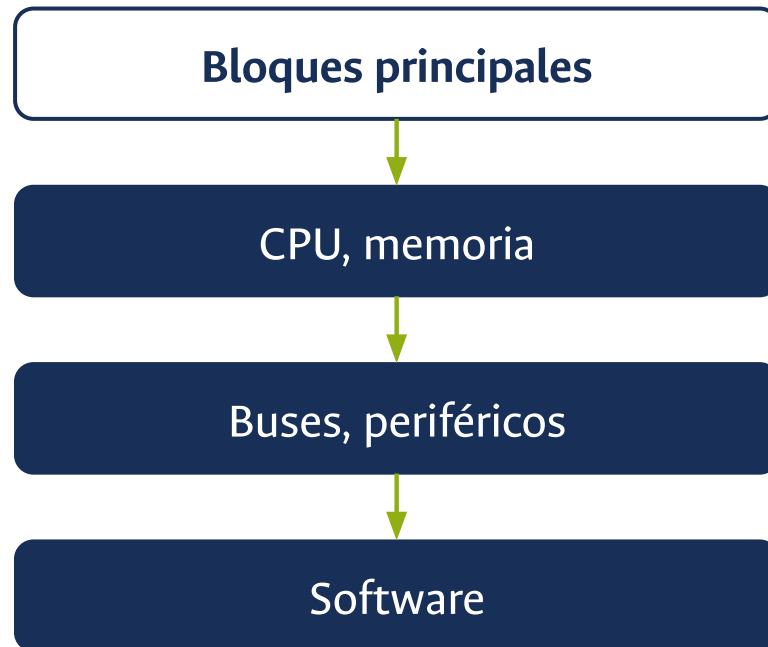


## ¿Qué falta para construir un computador completo?

# ¿Qué es realmente un computador?



Un computador está moderno está compuesto por:



La arquitectura define el modelo de programación.

## Arquitectura vs Microarquitectura

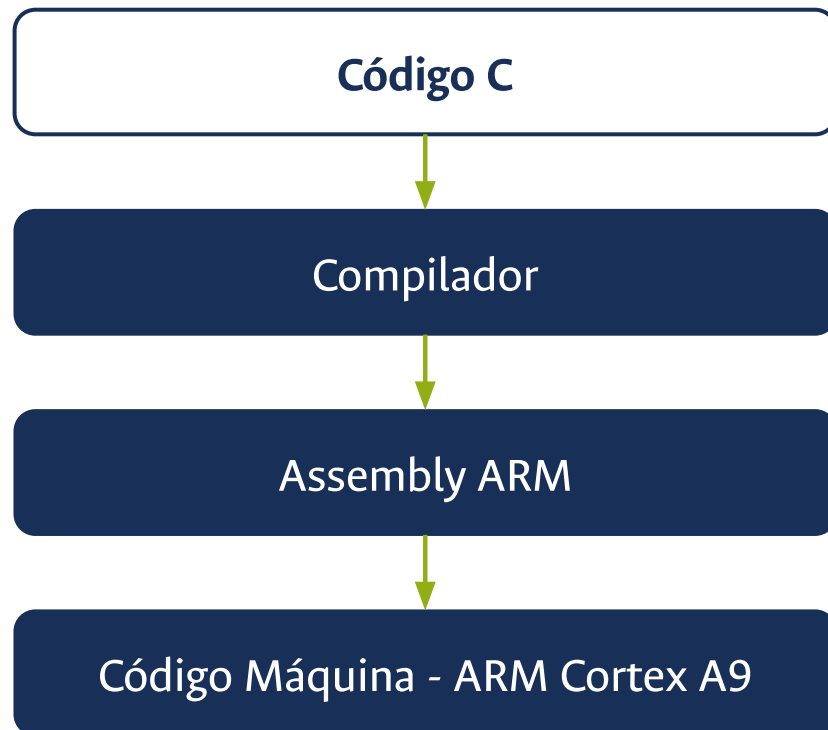
| Arquitectura  | Microarquitectura |
|---------------|-------------------|
| Qué hace      | Cómo lo hace      |
| ISA           | Pipeline          |
| Registros     | ALU interna       |
| instrucciones | Caché             |

La microarquitectura define cómo el hardware ejecuta internamente las instrucciones.



¿Qué es Assembler y cómo ejecuta instrucciones un procesador?

El procesador NO entiende C



Ejemplo código C

```
led = 1;
```

Assembly ARM

```
MOV R0, #1  
STR R0, [R1]
```

Assembler representa directamente las instrucciones que ejecuta físicamente el procesador.

# Comunicación PS-PL y buses AXI

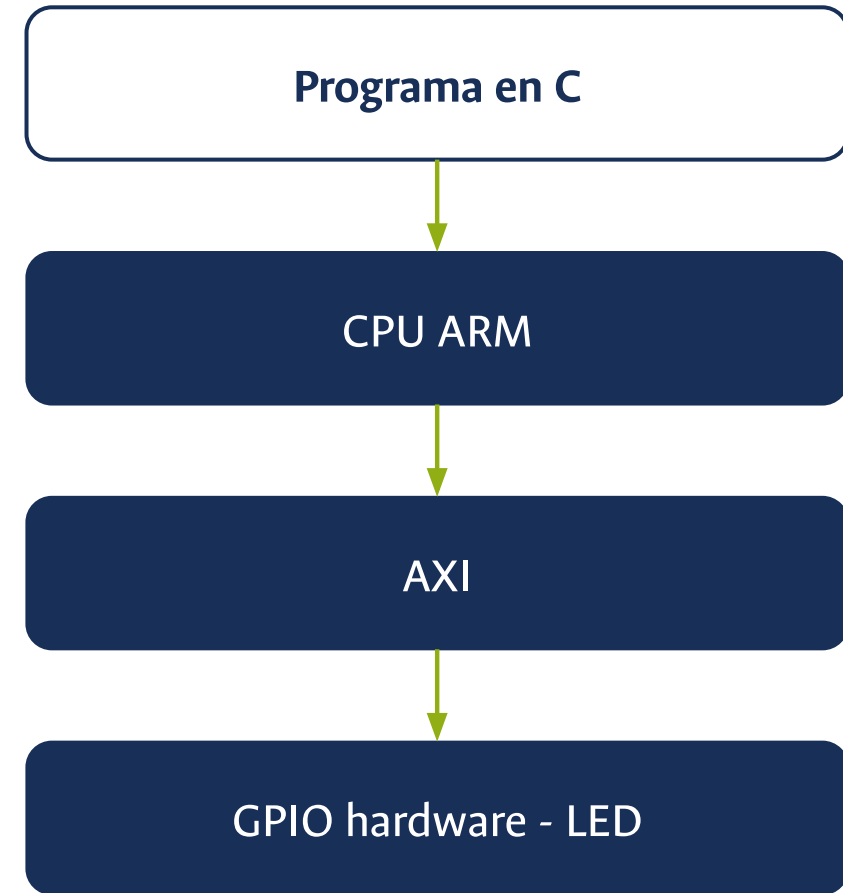
¿Cómo controla el procesador al hardware?  
Bus AXI

Permite:

- lectura/escritura
- memoria mapeada
- acceso a periféricos
- integración hardware/software

Conceptos importantes:

- AXI interconnect /GPIO
- Memory Mapping
- BRAM
- Address Editor



Los periféricos aparecen como direcciones de memoria accesibles desde el software



# ¿Qué hacen Vivado y Vitis?

| Vivado       | Vitis        |
|--------------|--------------|
| Hardware     | Software     |
| FPGA         | ARM          |
| Block Design | Aplicación C |
| Bitstream    | Ejecutable   |
| RTL/AXI      | Drivers/API  |

Vivado construye el sistema físico  
Vitis programa el comportamiento del sistema





# ¿Qué estamos haciendo realmente?

## Hardware:

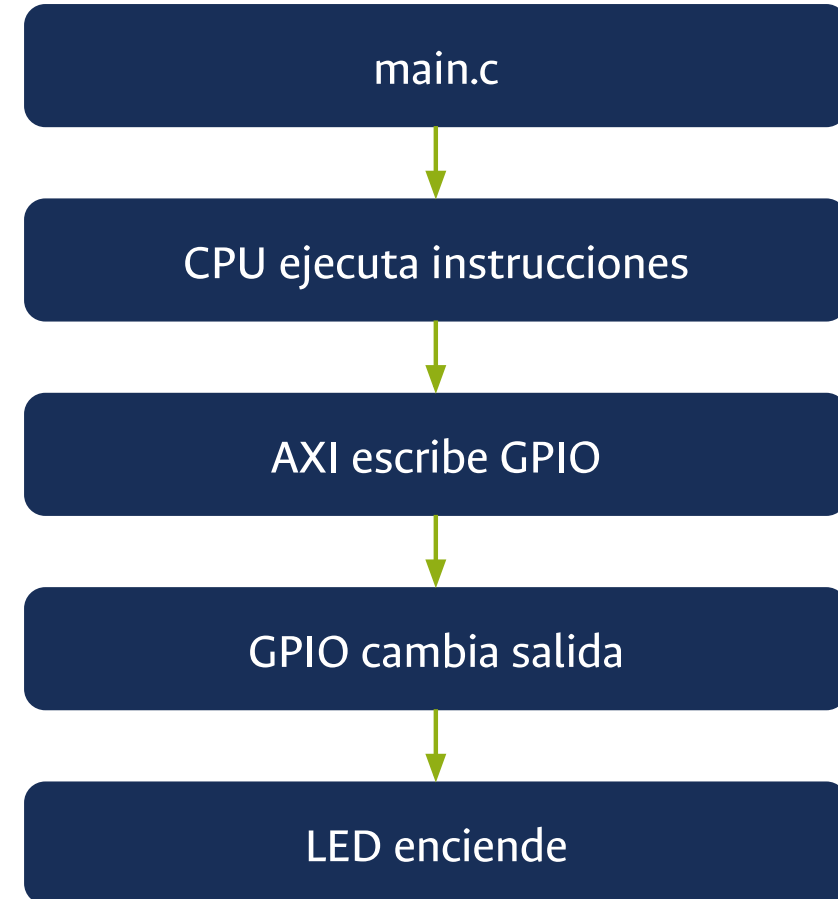
- creando un SoC
- conectando CPU y FPGA
- integrando periféricos
- creando buses AXI

## Software:

- ejecutando código en ARM
- controlando hardware desde C
- interactuando con GPIO

El laboratorio implementa un sistema computacional completo sobre FPGA

## Flujo del LED



# Conclusiones y proyección

## Lo aprendido:

- Arquitectura SoC
- Integración hardware/software
- Comunicación PS-PL
- Sistemas embebidos
- Flujo Vivado + Vitis

## Proyección industrial:

- Apple Silicon
- NVIDIA Jetson
- AMD Ryzen AI
- Sistemas aeroespaciales
- Edge AI
- Robótica

La FPGA deja de ser únicamente lógica digital y se convierte en un sistema computacional programable completo.

## Relación con el resto del curso





# Colores

## Facultad de Ingeniería



## Departamento de Ingeniería Eléctrica y Electrónica



## Colores por programa



Pregrados



Posgrados



Especialización

Maestría

Doctorado



# ELEMENTOS



